ENGINEERING ROADMAP / WINDOWS LOCAL-FIRST

Helpyard
One-Click Startup & Reliability Upgrade

Evidence-based implementation plan for a single-click Windows launch, automatic browser connection, complete section validation, and dependable background services.

Recommendation: keep the existing HTML/CSS/JavaScript UI and native PowerShell bridge. Add one double-click Windows **.bat** launcher, a safe in-memory connection bootstrap, and concurrent/bounded bridge request handling. No Python, pip, Java, or frontend rewrite is currently needed.

Prepared: 17 October 2026
Repository: khuttes/MCP (local workspace)
Baseline: Windows 11, Windows PowerShell 5.1, Python 3.14, Node.js 24, HTML/CSS/JavaScript dashboard

Scope: plan and test assessment only. No launcher/backend feature implementation is included in this deliverable.

INTERNAL WORKING DOCUMENT | Local workstation control remains loopback-only.

1. Executive summary

This plan is grounded in current repository files and tests run on this Windows workstation. It distinguishes observed behavior from untested or intentionally disabled features.

The project is a single-page HTML/CSS/JavaScript dashboard connected to a Windows PowerShell HttpListener bridge. The bridge now also serves the dashboard, so a separate Python static server is unnecessary. The latest baseline code checks and all 12 navigation destinations pass. Live testing verified system metrics, project discovery, process/service lists, local Ollama models and one chat response.

Reliability issue found in live browser testing: during a slow local Ollama completion, the synchronous bridge handler delayed health pings long enough for the dashboard to mark itself Offline and show a 15-second ping timeout. The local AI reply eventually succeeded. The bridge needs bounded concurrent request handling (or equivalent non-blocking design) before the one-click experience can be considered reliable.

Target user experience

- **Double-click one Start-Helpyard.bat file** from the project folder or shortcut.
- The launcher starts exactly one local bridge, waits for readiness, opens the correct dashboard URL even after a port fallback, and reports actionable startup failures.
- The dashboard authenticates to its own bridge without copying a token; the secret remains short-lived and in memory, never in the URL or localStorage.
- All dashboard sections load their supported features. Optional/missing Ollama or cloud credentials report status without making the core dashboard appear offline.
- A slow chat cannot block ping, metrics, or other browser requests; stop/restart behavior is documented and testable.

Area	Observed status	Evidence / boundary
Frontend structure	**Pass**	Inline JavaScript parses; 12 navigation targets, 78 DOM IDs, and 10 API route references match implemented bridge routes.
Bridge	**Pass with issue**	Windows PowerShell 5.1 parser clean; 29 bridge smoke assertions pass; synchronous long-running route blocks other requests.
Browser UI	**Pass with issue**	All 12 sections navigated; runtime lists and stats rendered; UI went Offline on ping timeout during Ollama inference.
Integrations	**Partial**	Ollama tags and test prompt worked on this host. Cloud APIs need user keys; unsafe shell/MCP startup intentionally disabled.
Packaging	**Not implemented**	No double-click .bat launcher; no package/dependency manifests or reproducible installer.

2. Audit scope and verification evidence

Check	Result	What was exercised
PowerShell syntax	**PASS**	Parsed bridge.ps1 with the installed Windows PowerShell 5.1 parser; no parse errors.
Bridge smoke suite	**PASS: 29 assertions**	HTML routes and unauthenticated probe; authorized ping/health/stats/read routes; invalid auth; CORS/Host defenses; denied unsafe operations; malformed JSON; unknown route.
Dashboard static checks	**PASS**	Inline JavaScript syntax; 12 navigation views; 78 referenced DOM IDs; 10 frontend-to-bridge API route references.
Live authenticated browser	**PASS with ping timeout**	Connected using a disposable local test credential; system values rendered; projects 1, processes 60, services 296; all 12 sections activated.

Check	Result	What was exercised
Local AI	**PASS with reliability defect**	15 Ollama models listed; a browser chat returned READY. Health ping timed out during the slow chat and UI reported Offline.
Portable/browser QA	**NOT RUN**	Additional Edge/Chrome/Firefox and clean-profile matrices not run; only the integrated Chromium browser was exercised.

Not performed for safety or access reasons: no restart/shutdown/sleep, process kill, service mutation, Claude configuration write, arbitrary shell execution, MCP startup, or cloud provider call was executed. Cloud APIs require a user-owned credential. The terminal and MCP start paths are intentionally disabled. Ollama/chat testing used a harmless one-word prompt.

Reviewed artifacts: bridge.ps1, pc-control-dashboard.html, tests/bridge-smoke.ps1, tests/dashboard-check.js, README.md, .vscode/launch.json, .vscode/tasks.json, Plannumber1.html, Project Report Number1.html and existing PDFs. No package.json, requirements.txt, pyproject.toml, or Java build manifest is present. README and Project Report Number1.html were updated during the preceding connection repair; older existing plan PDFs should be treated as superseded when this plan is approved.

3. Findings and priority order

Priority	Finding	Impact / planned response
P0	**Bridge handles requests serially.** Slow synchronous Ollama calls delay unrelated pings.	Confirmed by browser: the reply succeeded after the dashboard health ping timed out and marked the bridge Offline. Introduce bounded concurrency with per-request error isolation; add slow-route overlap and load tests.
P1	**No single double-click launcher.** Current startup requires a terminal/task and manual token entry.	Add a root-resolving .bat launcher to start one PowerShell bridge and open the correct dynamic-port URL; provide visible failure diagnostics and predictable stop behavior.
P1	**Connection bootstrap is not automatic.** The page can discover its bridge URL but still needs a pasted token.	Design a short-lived, origin-bound bootstrap that places credentials in browser memory only. Never put bearer tokens in URLs, command-line arguments, durable browser storage, or ordinary logs.
P1	**Several UI catches hide details.** Stats errors are swallowed and connectivity can be mislabeled.	Give sections independent loading/ready/offline states; show exact actionable failures without treating optional integration failures as bridge failure.
P2	**Coverage does not prove every user action.** Existing tests cover contracts and smoke APIs, not all interactive flows.	Add an end-to-end matrix for launch/port fallback, reconnect, every navigation section, filters/refresh, optional integrations, and approved-action denials.
P2	**Docs and packaging can drift.** Several reports predate the latest one-origin bridge design.	Update runbook, release checklist, and companion report after implementation; state supported Windows/browser versions and optional setup exactly.

Security guardrails that remain mandatory

- Keep the listener on IPv4 loopback. Never bind to a LAN/public interface or describe this privileged local bridge as a public API.
- Use a fresh, high-entropy session credential; scope bootstrap to the expected local browser origin, expire/revoke it on shutdown, and test hostile-origin and replay cases.
- Do not store the bearer credential in localStorage/sessionStorage, URLs, source, .bat text, process arguments, persistent logs, or crash diagnostics.
- Keep risky changes behind bridge-side approval. Do not implement arbitrary PowerShell execution or enable disabled MCP process startup as part of the launcher work.
- Use bounded request concurrency/queue capacity and timeouts; reject overload cleanly instead of spawning unlimited workers.

4. Recommended architecture and choices

Keep the existing stack for this reliability upgrade. PowerShell already owns Windows metrics/actions and localhost HTTP; replacing it with Python or Java would add a second runtime, packaging/update obligations, and an OS-control integration boundary without addressing the diagnosed serial-request defect. Python 3.14 and ReportLab happen to be installed locally for document generation, but are not application prerequisites.

Layer	Recommended responsibility	Acceptance
Single launcher	**Start-Helpyard.bat** resolves its own directory and invokes a PowerShell entry script with a process-scoped execution-policy bypass.	Double-click from Explorer and paths with spaces work; no working-directory assumption, admin prompt, or pip install.
Bridge lifecycle	Own one bridge process; select a free loopback port; validate the HTML file; wait on a readiness endpoint; emit useful diagnostics; close cleanly.	No duplicate instance or orphan; port fallback used consistently; startup failures show their cause.
Dashboard	Continue serving inline HTML/CSS/JavaScript from the bridge; use actual serving origin for every API request.	No separate port 8000 server, no stale default port, no cross-origin setup required.
Credential handoff	Use a one-time, short-lived local bootstrap bound to same-origin context; load token into JS memory and revoke it on shutdown.	No copy/paste for the normal one-click flow; static source and browser storage contain no token; abuse/replay checks pass.
Request serving	Handle independent bridge requests concurrently with a bounded worker/queue and safe per-request context ownership.	A slow local AI request never blocks ping/metrics; overload/timeouts return clear JSON errors.
Optional integrations	Detect Ollama and cloud setup separately; keep dashboard core healthy when optional providers are offline/unconfigured.	Local/cloud status is accurate; no provider key is required to launch or browse other sections.

Windows naming: use a **.bat** (or .cmd) file for double-click launch in this Windows workspace; ?bash? is not a native Windows batch file. It can invoke PowerShell internally. The frontend stays HTML/CSS/JS; Java and new Python packages are not recommended for this goal.

5. Step-by-step implementation plan

Phase 0 - Freeze the baseline and capture evidence (P0 | prerequisite)

- Record current parser/test/browser results and supported runtime versions.
- Keep current HTML/CSS/JS and PowerShell architecture; treat Ollama/cloud as optional integrations.
- Define expected startup, authentication, bridge status, and stop semantics before altering lifecycle code.

Gate: Baseline report and clear pass/fail acceptance criteria; no unrelated redesign.

Phase 1 - Fix bridge responsiveness and error isolation (P0 | critical)

- Refactor the listener to dispatch work through a bounded, tested concurrency design compatible with Windows PowerShell 5.1.
- Preserve request-specific origin/host/auth checks, response headers, bounded body sizes, stable JSON errors, and clean shutdown under concurrency.
- Remove swallowed errors in stats/connection refresh paths and present precise statuses.

Gate: While an Ollama/chat route is slow, repeated ping/stats/other reads still complete within target latency; no request mix-up or unhandled worker failure.

Phase 2 - Build the one-click launcher and lifecycle (P1 | after Phase 1)

- Add Start-Helpyard.bat at project root; resolve script paths from %~dp0 so any working directory/space in path works.
- Start bridge in the correct user session, wait for its ready endpoint, then open its printed Dashboard URL exactly once.
- Handle port fallback, missing PowerShell/dashboard, duplicate start, browser-launch failure, user stop, and restart without leaving orphan services.

Gate: A clean Windows user can double-click the file and get an actionable outcome; no Python/pip/Java install and no admin elevation by default.

Phase 3 - Secure automatic browser connection (P1 | security gate)

- Remove the manual token copy requirement only through a reviewed local one-time bootstrap handshake.
- Restrict the handshake to the bridge-served page/local origin; expire/revoke on use and shutdown; keep credentials only in JS memory.
- Add negative coverage for foreign origin, replay, expiry, missing token, wrong token, and reload/reconnect.

Gate: The ordinary launch reaches Connected without exposing a credential in URL, command arguments, saved state, or logs.

Phase 4 - Validate and repair every dashboard section (P1 | feature-complete)

- Add browser checks for all 12 navigable sections and every enabled read/refresh/filter action.
- Verify Settings reconnect, theme, workbench state, process/service read lists, projects, security, and disabled terminal/MCP behavior.
- Report Ollama unavailable/slow and missing cloud credentials independently from bridge availability; cloud calls remain opt-in and credential-backed.

Gate: Every enabled section renders correct data/state; disabled features remain clearly disabled; no stale ?Connected/Offline? contradiction.

Phase 5 - Full test, QA matrix, docs and release gate (P1 | sign-off)

- Run PowerShell parser and complete API-negative suite on Windows PowerShell 5.1; test PowerShell 7 only if declared supported.
- Run static frontend suite and real browser navigation/e2e suite in supported Chromium browsers; report Firefox/mobile as untested until actually tested.
- Stress slow routes/concurrent clients, port collision, restart/stop, corrupt browser state, browser disconnect, and unavailable optional services.
- Update README, VS Code task/launch docs, project report, and release notes; retain reproducible test output without credentials.

Gate: All P0/P1 acceptance gates pass; limitations and supported environments are stated honestly.

6. End-to-end test and acceptance matrix

Test group	Scenarios	Pass condition
Startup / packaging	Double click from Explorer; paths with spaces; occupied preferred port; missing file/runtime; duplicate launcher; open/close/restart.	Dashboard opens at the printed actual origin; one service instance; failures visible; clean stop.
Authentication	Valid, missing, wrong credential; one-time issue/use; foreign Origin/Host; replay; expiry; browser reload and bridge restart.	Valid session connects; all unauthorized cases rejected; no credential in URL, saved storage or logs.
Responsive server	Slow Ollama completion plus concurrent ping/stats/projects/process/services; bounded burst; client disconnect; worker exception.	Slow request does not block health; requests remain isolated; overload gives controlled status; no stuck/orphan worker.
Twelve UI sections	Overview, Projects, Workbench, Processes, Services, Terminal, Local AI, Cloud AI, MCP, Skills, Security, Settings.	Correct active view; every supported data source updates; placeholders/disabled features are explicit; browser console has no app errors.
Read routes	ping, health, stats, actions, projects, processes, services, MCP status, Ollama tags, Claude config.	Correct schema/status for authenticated requests; no secret leakage; unavailable optional integration is distinguishable.
Mutation/negative	Malformed/oversized JSON, unknown path/action, absent approval, wrong approval type, non-admin operation, hostile host/origin.	Stable 4xx JSON errors; no system mutation on denial; no arbitrary shell execution.
Optional integrations	Ollama absent/present/slow; empty models; chat failure; cloud key absent, invalid, provider unavailable.	Core dashboard stays connected; optional status/error is explicit; secrets not persisted.
Browser/platform	Declared supported Windows versions/browsers, fresh profile, narrow viewport, reload/offline/reconnect.	Versioned results and screenshots/logs without tokens; untested browsers remain labeled untested.

Release acceptance checklist

- A non-admin double-click start reaches a usable Connected dashboard from one browser tab.
- All 12 section controls work; optional integrations can be offline without breaking local dashboard/core APIs.
- Concurrent bridge requests remain responsive while local chat runs; automated test reliably catches regressions.
- Launcher handles port conflict, repeated launch, missing files, and graceful stop with clear diagnostics.
- All automated tests pass; cross-browser evidence is recorded; no unresolved P0/P1 reliability or security issue.
- No Python/pip/Java dependencies are installed for the product unless a later measured need justifies them.

7. Decisions required before implementation

The user requested a plan-first workflow. This document stops before the broader implementation because the automatic credential handshake and background-process lifecycle materially affect security and how startup/stop should work. The recommendation is to proceed with the defaults below; please confirm or change them before implementation.

Decision	Recommended default	Alternative / trade-off
Backend/runtime	**Keep PowerShell**; no Python/Java package migration.	Python rewrite adds pip/runtime/package handling and Windows OS adapters; Java is not warranted for a Windows-only local dashboard.
Launcher	**One root Start-Helpyard.bat** that starts the bridge, waits ready, then opens the page.	Keep VS Code task as developer option; create a desktop shortcut later if desired.
Auto-auth	**One-time short-lived same-origin bootstrap**; token stays only in browser memory.	Manual token paste is simpler but does not meet automatic connection request.

Decision	Recommended default	Alternative / trade-off
Background window	**Keep a visible/minimized console** with startup diagnostics; Ctrl+C stops the server.	Fully hidden mode feels simpler but complicates diagnostics and stopping/restarting background processes.
Scope of next pass	**Reliability, launcher, auth, all-section tests only.**	Defer new product panels/3D editor, unrestricted terminal, MCP startup, and public/LAN use.

Go / no-go: do not install dependencies or migrate the frontend/backend for appearance alone. Proceed after choosing the implementation defaults, and keep all workstation-control endpoints local-only. ?100% error-free? cannot be promised, but these test gates make failures detectable, diagnosable, and repeatable.

Plan status: awaiting implementation preference/approval. This PDF records verified baseline evidence as of 17 October 2026; future implementation claims must be backed by a new test run.